<u>מבוא לבינה מלאכותית</u>

סמסטר חורף תשפ"ז

<u>מטלה 1</u>

תאריך הגשה: 23:55 23.11.26

הנחיות

- שאלות בנושא מטלה זו יש לשאול דרך המודל, בפורום "מטלה 1".
- הוראות להגשת המטלה מופיעים בסוף מסמך זה.
- הקבצים הנדרשים להרצת הקוד הינם:
  - heuristics.py
  - main.py
  - flat_board_state.py
  - search.py
  - search_node.py
- העבודה להגשה בזוגות בלבד אלא אם כן המגישים קיבלו אישור להגשה שאינה בזוגות.
- לפני שניגשים לממש את המטלה מומלץ לעיין רבות בהסברים וכן בקוד הקיים.
- פתרון המטלה שתגישו ייבדק מול שאר ההגשות על ידי תוכנת העתקות.
- **מי שימצא כי העתיק יכשל בקורס וכן יועבר לוועדת משמעת אוניברסיטאית**.
- הפרויקט נכתב וייבדק בשפת התכנות python3.10.
- מסמך זה בנוי באופן הבא: תיאור המטלה, בעיית החיפוש, מרחב הבעיה, שאלות המטלה, פונקציות שיש להשלים והסבר על פונקציות קיימות.
- במידה ויש בעיה עם הטסטים או כל בעיה גורפת בבדיקות ניתן לבקש מהועד ליצור קשר עם בודק התרגילים – דניס denisst@post.bgu.ac.il

תיאור המטלה

המשחק "לוח-שטוח" הינו משחק לוח-משבצות המכיל קירות, משבצות מלאות ומשבצות חסרות (חורים). על חלק מהמשבצות המלאות מונחות קוביות. כמות הקוביות שווה לכמות המשבצות החסרות. בכל צעד במשחק ניתן לבחור פעולה אחת מתוך 4 פעולות אפשריות: למעלה, למטה, ימינה או שמאלה. בכל פעם שנבחרת פעולה, כל הקוביות צריכות לבצע את הצעד שהפעולה הגדירה. קוביה תחליק צעד אחד בכיוון שהוגדר בפעולה אם קיימת משבצת פנויה שהיא לא קיר ולא קוביה אחרת שלא יכולה לזוז. במידה וקוביה החליקה אל חור, היא סותמת את החור, נעלמת והמשבצת הופכת להיות משבצת מלאה. אם הקוביה מנסה לזוז בכיוון שהוגדר אל תוך קיר, קובייה אחרת שלא יכולה לזוז או מעבר לקצה הלוח, הקובייה תשאר במקום. מטרת המשחק היא לסתום את כל החורים של הלוח בעזרת הקוביות בכמה שפחות צעדים. ממשו את אלגוריתם **A*** כדי למזער מספר הפעולות ולפתור את המשחק.

בעיית חיפוש

בהינתן לוח המשחק, מיקומי הקוביות, הקירות והחורים, עליכם למצוא את רצף המצבים המינימלי שסותם את כל חורי המשחק.

מרחב הבעיה

מערך בגודל NxM כך שבכל תא יש ערך מ 0-3:

הערך 0 מציין שבתא חסרה משבצת (חור).

הערך 1 מציין שהתא מכיל משבצת.

הערך 2 מציין שהתא מכיל קיר.

הערך 3 מציין שהתא מכיל קובייה.

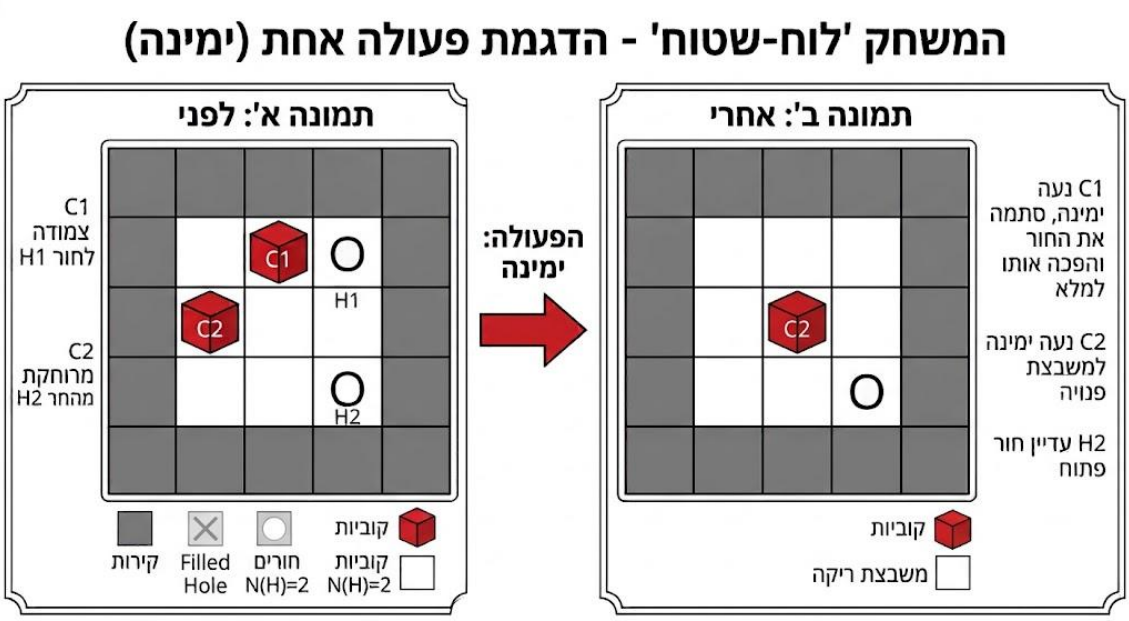
<u>מצב התחלתי</u>: מערך דו מימדי עם הערכים 0-3 בכל תא (לוח משחק נתון).

<u>מצב סופי</u>: לוח משחק שלא מכיל חורים.

<u>אופרטורים</u>:

4 אופרטורים: למעלה, למטה, ימינה ושמאלה.

בכל פעם שמבוצעת פעולה כל הקוביות חייבות לזוז בכיוון הפעולה משבצת אחת. ישנם שני מקרים בהם קוביה לא יכולה לזוז בכיוון הפעולה – אם היא מתנגשת בקיר או מתנגשת בקוביה אחרת שלא יכולה לזוז.



שאלות המטלה

א. השלימו את הפונקציות החסרות (מצויינות בהמשך).
ב. שנו את היוריסטיקה base heuristic כך שתחשב את היוריסטיקה הבאה- עבור כל משבצת ריקה נחשב את מרחק המנהטן הקצר ביותר מכל הקוביות אליה. נקח את הערך המקסימלי מבין כל המרחקים הקצרים לכל משבצת ריקה.

ג. שנו את היוריסטיקה advanced_heuristic כך שתחשב היוריסטיקה של מצב מסויים בבעיה. על היוריסטיקה לשפר את היוריסיטיקה מסעיף ב' (ימדד בזמן ריצה או בכמות קודקודים שפותחו או הורחבו) ולשמור על אופטימליות הפתרון.
הזוג שהריץ את הטסטים של סעיף ג' בזמן הממוצע הקצר ביותר (ימדד בהרצת הטסטים במודל ואולי בטסטים נוספים רק כדי להכריע את הבונוס), יקבל **2 נקודות בונוס לציון הסופי**.
שימו לב שכל הטסטים צריכים לעבור כדי שהזמן יחושב.
בנוסף, כדי למדוד את פונקציית היוריסטיקה בצורה מדוייקת יותר, הוספנו בטסטים מימוש של קובץ ה search ושם נקרא לפונקציות היוריסטיקה (והגדרת המצבים) שלכם כדי למדוד את זמן הריצה עם אותו מימוש של מבנה נתונים ולוגיקת חיפוש. גם כאן ינתנו **2 נקודות בונוס לציון הסופי.**
כמו כן, 2 קטגוריות נוספות יבדקו:

כמות הקודקודים עבורם עשיתם generate.

כמות הקודקודים עבורם עשיתם expand.

גם כאן ינתנו **2 נקודות בונוס לציון הסופי עבור כל אחת מהקטגוריות למי שזוכה.** שוויון ישבר לפי הזמן ריצה הממוצע המהיר יותר.

זוגות שרוצים להשתתף בבונוס צריכים לרשום את תעודות הזהות שלהם והזמן שקיבלו בהרצה בקובץ הזה.
(זוגות שלא רוצים להשתתף בבונוס צריכים לרשום רק את תעודת הזהות בלי הזמן שקיבלו)

עבור הבונוסים - במידה ויהיה קשה להבדיל בין זמני הריצה (למשל, כמה צוותים יקבלו זמן ריצה של 0.0) נריץ עבור צוותים אלו טסטים קשים יותר (לוח גדול יותר, יותר מכשולים וכו'..).

הטסטים הנוספים לא משפיעי על הציון והציון שאתם רואים במודל הינו הציון לעבודה ולא נבדוק אותכם יותר מכך.

הנחות שניתן להניח:

1. כמות החורים שווה לכמות הקוביות על הלוח.
2. הקוביה מונחת על משבצת מלאה במצב ההתחלתי.
3. דפנות הלוח הינן קיר גם אם המשבצת האחרונה אינה קיר.
4. הלוח המתקבל בהכרח מלבן או ריבוע (לא יכול להיות אורך שונה של שורה או עמודה).

הפונקציות שיש להשלים למימוש כל אחד מאלגוריתמי החיפוש

heuristics.py

base_heuristic(_flat_board_state)

הפונקציה מקבלת אובייקט מסוג flat_board_state ומחזירה את ערך היוריסטיקה עבור הלוח-שטוח בקלט. (יש למחוק את השורה שמחזירה 0)

advanced_heuristic (_flat_board_state)

הפונקציה מקבלת אובייקט מסוג flat_board_state ומחזירה את ערך היוריסטיקה עבור הלוח-שטוח בקלט. (יש למחוק את השורה שמחזירה 0)

flat_board_state.py

is_goal_state(_flat_board_state)

פונקציה סטטית המקבלת מצב של לוח-שטוח ומחזירה אמת אם המצב הינו מצב סופי, אחרת תחזיר שקר.

get_neighbors(self)

הפונקציה לא מקבלת קלט ומחזירה רשימה של סדורות (tuples) כך שבכל סדורה יש את אובייקט מסוג flat_board_state המייצג את אחד השכנים ועלות המעבר אל אותו שכן.

[(flat_board_state1, cost1), (flat_board _state2, cost2)...]

get_state_board (self)

הפונקציה לא מקבלת קלט ומחזירה את יצוג של לוח-שטוח בתור רשימה של רשימות כך שכל רשימה מייצגת שורה של ערכים מספריים לפי התאור. לצורך הבדיקה שלכם – תבדקו שהפונקציה מחזירה בדיוק את הלוח של המצב ההתחלתי עבור המצב ההתחלתי. פונקציה זו יכולה לשמש עבורכם לבדוק את הקוד. כמו כן, נשתמש בה בבדיקות נכונות של הקוד שלכם.

get_state_str(self)

הפונקציה לא מקבלת קלט ומחזירה את יצוג המחרוזת של מגדל הקוביות (אין צורך לשנות את הפונקציה) - פונקציה זו לא חובה למימוש. תוכלו להיעזר בה כדי לדבג את הקוד.

search.py

create_open_set()

הפונקציה אינה מקבלת ערכים, ומחזירה מבני נתונים מתאים לאלגוריתם החיפוש.

create_closed_set()

הפונקציה אינה מקבלת ערכים, ומחזירה מבני נתונים מתאים לאלגוריתם החיפוש.

add_to_open(vn, open_set)

הפונקציה מקבלת כקלט את מבני הנתונים open ומצב vn ומכניסה את vn ל-open.

open_not_empty(open_set)

הפונקציה מקבלת כקלת את מבני הנתונים open ומחזירה אמת אם הוא לא ריק, אחרת, מחזירה שקר.

get_best(open_set)

הפונקציה מקבלת כקלט את מבני הנתונים ומחזירה את הקודקוד הטוב ביותר, על פי אלגוריתם החיפוש.

add_to_closed(vn, closed_set)

הפונקציה מקבלת כקלט את מבני הנתונים closed ומצב vn ומכניסה את vn ל-closed .

duplicate_in_open(vn, open_set)

הפונקציה מקבלת כקלט את מבני הנתונים open ומצב vn ומחזירה אמת אם קיים קודקוד ב open עם מצב זהה ל vn.state וערך ה g שלו קטן או שווה ל vn.g. אם vn.state לא קיים ב open או ערך ה g של המצב ב open גדול מ vn.g , הפונקציה תחזיר שקר.

duplicate_in_close(vn, closed_set)

הפונקציה מקבלת כקלט את מבני הנתונים closed ומצב vn ומחזירה אמת אם קיים קודקוד ב closed עם מצב זהה ל vn.state וערך ה g שלו קטן או שווה ל vn.g. אם vn.state לא קיים ב closed או ערך ה g של המצב ב closed גדול מ vn.g , הפונקציה תחזיר שקר.

מידע על פונקציות ממומשות

search_node.py

get_neighbors(self)

הפונקציה לא מקבלת קלט ומחזירה את כל המצבים השכנים של המצב הנוכחי.

search.py

print_path(path)

הפונקציה מקבלת מערך של קודקודים מסוג search_node ומדפיסה את המחרוזת המייצגת את המצב של מגדל הקוביות.

search(start_state, heuristic)

הפונקציה מקבלת את המצב ההתחלתי, פונקצית היוריסטיקה ומחזירה מערך של search_node המייצגים את המסלול האופטימלי מהמצב ההתחלתי למצב הסופי. בתא הראשון במערך יהיה המצב ההתחלתי בתא הבא יהיה השכן שלו במסלול שהוחזר וכך הלאה עד המצב הסופי בתא האחרון.

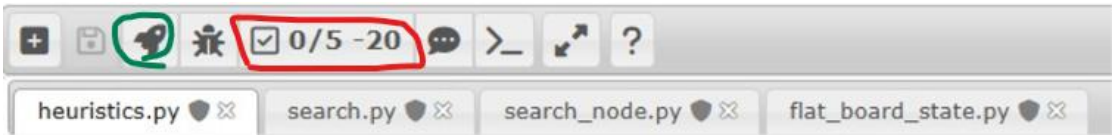
הגשת המטלה

- יש להגיש דרך מערכת המודל את ארבעת הקבצים הבאים כקובץ ZIP:
  - heuristics.py
  - flat_board_state.py
  - search.py
  - search_node.py

הגשת המטלות תתבצע ישירות מול מערכת המודל בצורה אלקטרונית.

שימו לב: ניתן לנסות לקבל ציון בVPL **5 פעמים בלבד.** לאחר מכן ירד ציון משמעותי מהמטלה.

עם זאת, לאחר שהעלתם את הקבצים, ניתן להריץ בדיקה עם טסטים בסיסיים (ללא ציון) ולראות שהכל עובד כשורה.



את הכפתור בעיגול הירוק אתם יכולים להריץ כמה פעמים שתרצו ולוודא שהקוד תקין לפני הרצה של קבלת ציון. במידה והכל תקין – תריצו את הבדיקה שתתן ציון (מוקף בעיגול אדום).

ניתן לראות את הפידבק להרצה (קומפילציה, מספר טסטים שעברו, שגיאות זמן ריצה וכו'...).

לאחר סיום ההרצה יתקבלו התוצאות. ישנם מספר טסטים הבודקים את הפתרון המוגש למטלה. אם התקבלה שגיאת קומפליציה יש עליכם להעלות קובץ חדש בכדי לקבל ציון לאחר התיקונים. בקבצים המקוריים שקיבלתם עם המטלה עלולות להיות שגיאות ריצה עבור פונקציות לא ממומשות.

טיפים לעבודה

במידה שהטסטים ב VPL לא עוברים, אין סיבה להסיק שיש בעיה בטסטים (זה יכול לקרות אך הסבירות נמוכה). יש לבדוק את הקוד שלכם קודם:

- האם הרצתם דוגמאות פשוטות ב main?
- האם וידאתם שהיוריסטיקה שלכם מחזירה את הערך שציפיתם לו?
- האם בדקתם את תקינות היוריסטיקה המתקדמת?
- אם יש לכם בעיות זמן ריצה- האם השתמשתם במבני נתונים יעילים? למשל- מה הסיבוכיות של בדיקת כפילויות ב OPEN? האם אפשר לייעל?

בהצלחה!